Passing Data to Views

In the last lesson, we saw how to create a basic view. We haven't even gotten into Blade templates yet, but before we do that, I want to look at the different ways that you can pass data into a view. This is something that you will be diong a lot. For instance, if you fetch some data from your database through the model, you will want to pass that data to the view to display it.

This works the same way whether you are using Blade templates or plain PHP files. Let's look at how to pass data to a view using multiple methods.

Pass An Array

You can pass data to views by passing an associative array as the second argument to the `view()` function.

Let's pass in a title to the view:

```
Route::get('/jobs', function () {
    return view('jobs', [
        'title' => 'Available Jobs',
    ]);
});
```

Now, let's update our view to display the title:

```
  <h1><?php echo $title; ?></h1>
```

`->with()` Method Alternative

You can also use the `->with()` method to pass data to views. The `->with()` method is a more fluent way of passing data to views. Here is an example:

```
Route::get('/jobs', function () {
    return view('jobs')->with('title', 'Available Jobs');
});
```

Passing an Array to Views

You can also pass arrays to views. Let's pass an array of jobs to the view:

```
Route::get('/jobs', function () {
    $jobs = [
        'Software Engineer',
        'Web Developer',
```

```
        'Data Scientist',
    ];

    return view('jobs', [
        'title' => 'Available Jobs',
        'jobs' => $jobs,
    ]);
});
```

You can also use the `->with()` method to pass an array to views. Here is an example:

```
Route::get('/jobs', function () {
    $jobs = [
        'Software Engineer',
        'Web Developer',
        'Data Scientist',
    ];

    return view('jobs')->with('title', 'Available Jobs')->with('jobs', $jobs);
});
```

`compact()` Function

You can also use the `compact()` function to pass variables to views. The `compact()` function creates an array from the variable names you pass to it. Here is an example:

```
Route::get('/jobs', function () {
    $title = 'Available Jobs';
    $jobs = [
        'Software Engineer',
        'Web Developer',
        'Data Scientist',
    ];

    return view('jobs', compact('title', 'jobs'));
});
```

This is all preference. What I like to do is to use `->with` if there is only a single variable passed and `compact()` if there are multiple variables passed. Again, this is completely up to you and there is no right or wrong way to do it.

Displaying Data in Views

Now, let's update our view to display the jobs:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Jobs List</title>
  </head>

  <body>
    <h1><?php echo $title; ?></h1>
    <ul>
      <?php foreach ($jobs as $job) : ?>
      <li><?php echo htmlspecialchars($job, ENT_QUOTES, 'UTF-8'); ?></li>
      <?php endforeach; ?>
    </ul>
  </body>
</html>
```

As you can see, we have passed an array of jobs to the view and then looped through the jobs in the view to display them.

We used the `htmlspecialchars()` function to escape the output. This is to prevent XSS attacks. When we move to using Blade templates, we won't have to worry about escaping output, as Blade does this for us automatically. There are a lot of security features that not only Blade, but Laravel itself provides out of the box. We will look into these more later in the course.

In the next lesson, we will look into Blade templates and how to use them to render views.